BlockQuicksort: Avoiding Branch Mispredictions in Quicksort

STEFAN EDELKAMP, King's College London
ARMIN WEIß, Universität Stuttgart

It is well known that Quicksort – which is commonly considered as one of the fastest in-place sorting algorithms – suffers in an essential way from branch mispredictions. We present a novel approach to addressing this problem by partially decoupling control from dataflow: in order to perform the partitioning, we split the input into blocks of constant size. Then, all elements in one block are compared with the pivot and the outcomes of the comparisons are stored in a buffer. In a second pass, the respective elements are rearranged. By doing so, we avoid conditional branches based on outcomes of comparisons (except for the final Insertionsort). Moreover, we prove that when sorting n elements, the average total number of branch mispredictions is at most $\epsilon n \log n + O(n)$ for some small ϵ depending on the block size.

Our experimental results are promising: when sorting random-integer data, we achieve an increase in speed (number of elements sorted per second) of more than 80% over the GCC implementation of Quicksort (C++ `std::sort`). Also, for many other types of data and non-random inputs, there is still a significant speedup over `std::sort`. Only in a few special cases, such as sorted or almost sorted inputs, can `std::sort` beat our implementation. Moreover, on random-input permutations, our implementation is even slightly faster than an implementation of the highly tuned Super Scalar Sample Sort, which uses a linear amount of additional space.

Finally, we also apply our approach to Quickselect and obtain a speed-up of more than 100% over the GCC implementation (C++ `std::nth_element`).

CCS Concepts: • **Theory of computation → Sorting and searching**;

Additional Key Words and Phrases: In-place sorting, Quicksort, branch mispredictions

ACM Reference format:
Stefan Edelkamp and Armin Weiß. 2019. BlockQuicksort: Avoiding Branch Mispredictions in Quicksort. *ACM J. Exp. Algorithmics* 24, 1, Article 1.4 (January 2019), 22 pages.
https://doi.org/10.1145/3274660

1 INTRODUCTION

Sorting a sequence of elements of some totally ordered universe remains one of the most fascinating and well-studied topics in computer science. Moreover, it is an essential part of many practical applications. Thus, efficient sorting algorithms directly transfer to a performance gain for many applications. One of the most widely used sorting algorithms is Quicksort, which was introduced by Hoare in 1961 [19, 21] and is considered to be one of the most efficient sorting algorithms. For sorting an array, it works as follows: first, it chooses an arbitrary pivot element and then rearranges

Authors' addresses: S. Edelkamp, Department of Informatics, King's College London, Strand Campus, Bush House London WC2B 4BG; email: stefan.edelkamp@kcl.ac.uk; A. Weiß, Institut für Formale Methoden der Informatik, Universität Stuttgart, Universitätsstr 38, D-70569 Stuttgart; email: armin.weiss@fmi.uni-stuttgart.de.

1084-6654/2019/01-ART1.4 $15.00
https://doi.org/10.1145/3274660

the array such that all elements smaller than the pivot are moved to the left side and all elements larger than the pivot are moved to the right side of the array. This is called *partitioning*. Then, the left and right side are both sorted recursively. Although its average[1] number of comparisons is not optimal ($1.38n \log n + O(n)$ vs. $n \log n + O(n)$ for Mergesort), its overall instruction count is very low. Moreover, by choosing the pivot element as the median of a larger sample, the leading term $1.38n \log n$ for the average number of comparisons can be made smaller – even as low as $n \log n$ when choosing the pivot as the median of a sample of growing size [32]. Other advantages of Quicksort are that it is easy to implement and that it does not need extra memory except for the recursion stack of logarithmic size (even in the worst case if properly implemented). A major drawback of Quicksort is its quadratic worst-case runtime. Nevertheless, there are efficient ways to circumvent a really bad worst case. The most prominent is Introsort (introduced by Musser [33]) which is applied in GCC implementation of `std::sort`: as soon as the recursion depth exceeds a certain limit, the algorithm switches to Heapsort.

Another deficiency of Quicksort is that it suffers from branch mispredictions (or branch misses) in an essential way. On modern processors with long pipelines (14 stages for Intel Haswell, Broadwell, Skylake, Kaby Lake processors – for the older Pentium 4 processors even more than twice as many) every branch misprediction causes a rather long interruption of the execution since the pipeline has to be filled anew. Kaligosi and Sanders [22] analyzed the number of branch mispredictions incurred by Quicksort. They examined different simple branch prediction schemes (static prediction and 1-bit, 2-bit predictors) and showed that with all of them, Quicksort with a random element as pivot causes, on average, $cn \log n + O(n)$ branch mispredictions for some constant $c = 0.34$ (resp., $c = 0.46$, $c = 0.43$). In particular, in Quicksort with a random pivot element, every fourth comparison is followed by a mispredicted branch. The reason is that, for partitioning, each element is compared with the pivot and, depending on the outcome, either it is swapped with some other element or not. Since for an optimal pivot (the median), the probability of being smaller than the pivot is 50%, there is no way to predict these branches.

Kaligosi and Sanders also established experimentally that choosing skewed pivot elements (far off the median) can even decrease the runtime because it makes branches more predictable. This also explains the following: although theoretically larger samples for pivot selection were shown to be superior, in practice, the median-of-three variant turned out to be the best. The skewed pivot phenomenon is confirmed by Biggar et al. [7]. Moreover, Martínez et al. [29] give precise theoretical bounds on the number of branch misses for Quicksort – also establishing theoretical superiority of skewed pivots under the assumption that branch mispredictions are expensive.

Brodal and Moruz [9] proved a general lower bound on the number of branch mispredictions given that every comparison is followed by a conditional branch that depends on the outcome of the comparison. In this case, there are $\Omega(n \log_d n)$ branch mispredictions for a sorting algorithm that performs $O(dn \log n)$ comparisons. As Elmasry and Katajainen [12] remarked, this theorem no longer holds if the results of comparisons are not used for conditional branches. They showed that every program can be transformed into a program that induces only a constant number of branch misses and whose runtime is linear in the runtime of the original program. However, this general transformation introduces a huge constant factor overhead. Still, Elmasry, Katajainen, and Stenmark [12, 13] showed how to efficiently implement many algorithms related to sorting with only a few branch mispredictions. They call such programs *lean*. In particular, they present variants of Mergesort and Quicksort suffering only very little from branch misses. Their Quicksort variant (called Tuned Quicksort; for details on the implementation, see [23]) is

[1]Here and in the following, the average case refers to a uniform distribution of all input permutations assuming that all elements are different.

very fast for random permutations. However, it does not behave well with duplicate elements because it applies Lomuto's unidirectional partitioner (see, e. g., [10]).

Another development in recent years is multi-pivot Quicksort (i.e., several pivots in each partitioning stage [4, 5, 27, 38, 39]). It started with the introduction of Yaroslavskiy's dual-pivot Quicksort [41] that, surprisingly, was faster than known Quicksort variants and, thus, became the standard sorting implementation in Oracle Java 7 and Java 8. Concerning branch mispredictions, all of these multi-pivot variants behave essentially like ordinary Quicksort [29]. However, they have one advantage: every data element is accessed less frequently (this is also referred to as the number of *scans*). As outlined by Aumüller et al. [5], increasing the number of pivot elements further (up to 127 or 255), leads to Super Scalar Sample Sort, which has been introduced by Sanders and Winkel [35]. Super Scalar Sample Sort not only has the advantage of few scans but also is based on the idea of avoiding conditional branches. The correct bucket (the position between two pivot elements) can be found by converting the results of comparisons to integers and then simply performing integer arithmetic. In their experiments, Sanders and Winkel show that Super Scalar Sample Sort is approximately twice as fast as Quicksort (`std::sort`) when sorting random-integer data. However, Super Scalar Sample Sort has one major drawback: it uses a linear amount of extra space (for sorting n data elements, it requires space for other n data elements and additionally for more than n integers). In their conclusion, Kaligosi and Sanders [22] raised the question:

> *However, an in-place sorting algorithm that is better than Quicksort with skewed pivots is an open problem.*

(Note that there are different interpretations of in place, resp., in situ: for us, in place means that the algorithm needs only a constant or logarithmic amount of extra space. Thus, in place in our understanding certainly implies in place in the understanding of Kaligosi and Sanders [22].) In this work, we solve the problem by presenting our block partition algorithm, which allows implementation of Quicksort without any branch mispredictions incurred by conditional branches based on results of comparisons (except for the final Insertionsort – also, there are still conditional branches based on the control flow, but their number is relatively small). We call the resulting algorithm BlockQuicksort. Our work is inspired by Tuned Quicksort from [13], from which we also borrow parts of our implementation. The difference is that by doing the partitioning block-wise, we can use Hoare's partitioner, which is far better with duplicate elements than Lomuto's partitioner (although Tuned Quicksort can be made working with duplicates by applying a check for duplicates similar to what we propose for BlockQuicksort as one of the further improvements in Section 3.2). Moreover, BlockQuicksort is also superior to Tuned Quicksort for random permutations of integers.

A problem closely related to sorting is selection: in addition to the array, an index k is also given as part of the input – the task is to find the kth smallest element of the array in sorted order. Hoare's Quickselect [20], which works almost the same way as Quicksort, solves this problem. The difference between Quickselect and Quicksort is that after partitioning for Quickselect the recursion is only on the side where the kth element lies, while Quicksort recurses on both parts of the array. Because of this similarity, we can also apply our block partition algorithm to Quickselect and obtain BlockQuickselect.

Our Contributions.

- We present a variant of the partition procedure that only incurs few branch mispredictions by storing results of comparisons in constant size buffers.
- We prove an upper bound of $\epsilon n \log n + O(n)$ branch mispredictions, on average, where $\epsilon < \frac{1}{16}$ for our proposed block size (Theorem 3.1).

- We propose some improvements over the basic version.
- We implemented our algorithm with an `stl`-style interface[2].
- We conduct experiments and compare BlockQuicksort with classical Quicksort (`std::sort`), Yaroslavskiy's dual-pivot Quicksort and Super Scalar Sample Sort – on random-integer data it is faster than all of these and also Tuned Quicksort by Katajainen et al.
- We apply our partition procedure to Quickselect. In our experiments, we see an even larger speedup over the GCC implementation of Quickselect (`std::nth_element`) than in the case of sorting.

Outline. Section 2 introduces some general facts on branch predictors and mispredictions. It also contains a short account of standard improvements of Quicksort and a short description of Quickselect. In Section 3, we give a precise description of our block partition method and establish our main theoretical result – the bound on the number of branch mispredictions. Finally, in Section 4, we experimentally evaluate different block sizes and different pivot selection strategies, and compare our implementation with other state-of-the-art implementations of Quicksort and Super Scalar Sample Sort.

This work is an extended version of the conference paper [11]. Here, we present additional experimental results, including the implementation of Quickselect.

2 PRELIMINARIES

Logarithms denoted by log are always base 2. The term *average case* refers to a uniform distribution of all input permutations assuming that all elements are different. In the following, `std::sort` always refers to its GCC implementation.

Branch Misses. Branch mispredictions can occur when the code contains conditional jumps (i.e., *if* statements, loops, etc.). Whenever the execution flow reaches such a statement, the processor has to decide in advance which branch to follow and decode the subsequent instructions of that branch. Because of the length of the pipeline of modern microprocessors, a wrongly predicted branch causes a long delay since, before continuing the execution, the instructions for the other branch have to be decoded.

Branch Prediction Schemes. Precise branch prediction schemes of most modern processors are not disclosed to the public. However, the simplest schemes suffice to show that BlockQuicksort induces only a few mispredictions.

The easiest branch prediction scheme is the *static predictor*: for every conditional jump, the compiler marks the more likely branch. In particular, this means that for every *if* statement, we can assume that there is a misprediction if and only if the *if* branch is not taken. For every *loop* statement, there is precisely one misprediction for every time the execution flow reaches that loop – when the execution leaves the loop. For more information about branch prediction schemes, we refer to [18, Section 3.3].

How to Avoid Conditional Branches. The usual implementations of sorting algorithms perform conditional jumps based on the outcome of comparisons of data elements. There are at least two methods to avoid these conditional jumps – both are supported by the hardware of modern processors:

- Conditional moves (`CMOVcc` instructions on x86 processors) – or, more general, conditional execution. In C++, compilation to a conditional move can be (often) triggered by

```
i = (x < y) ? j : i.
```

[2]Code available at https://github.com/weissan/BlockQuicksort.

- Cast Boolean variables to integer (SETcc instructions x86 processors). In C++:

```
int i = (x < y).
```

Also, many other instruction sets support these methods (e.g., ARM [1], MIPS [34]). Still, the Intel Architecture Optimization Reference Manual [2] advises using these instructions only to avoid unpredictable branches (as is the case for sorting) since correctly predicted branches are still faster. For more examples on how to apply these methods to sorting, see [13].

Quicksort and Improvements. The central part of Quicksort is the partitioning procedure. Given a pivot element, it returns a pointer p to an element in the array and rearranges the array such that all elements left of the p are smaller or equal the pivot and all elements on the right are greater or equal the pivot. Quicksort first chooses a pivot element, performs the partitioning, and, finally, recurses on the elements smaller and larger than the pivot; see Algorithm 1. We call the procedure that organizes the calls to the partitioner the *Quicksort main loop*.

ALGORITHM 1: Quicksort

```
1: procedure QUICKSORT(A[ℓ, ..., r])
2:     if r > ℓ then
3:         pivot ← choosePivot(A[ℓ, ..., r])
4:         cut ← partition(A[ℓ, ..., r], pivot)
5:         Quicksort(A[ℓ, ..., cut − 1])
6:         Quicksort(A[cut, ..., r])
7:     end if
8: end procedure
```

There are many standard improvements for Quicksort. For our optimized Quicksort main loop (which is a modified version of Tuned Quicksort [13, 23]), we implemented the following:

- A very basic optimization due to Sedgewick [37] avoids recursion partially (tail recursion, e.g., `std::sort`) or totally (here, this requires the introduction of an explicit stack).
- Introsort [33]: There is an additional counter for the number of recursion levels. As soon as it exceeds some bound (`std::sort` uses $2 \log n$; we use $2 \log n + 3$), the algorithm stops Quicksort and switches to Heapsort [14, 40] (only for the respective sub-array). By doing so, a worst-case runtime of $O(n \log n)$ is guaranteed.
- Sedgewick [37] also proposed to switch to Insertionsort (see, e. g., [26, Section 5.2.1]) as soon as the array size is less than a fixed small constant (16 for `std::sort` and our implementation). There are two possibilities for when to apply Insertionsort: either during the recursion, when the array size becomes too small, or at the very end after Quicksort has finished. We implemented the first possibility (in contrast to `std::sort`) because for sorting integers, it hardly made a difference, but for larger data elements, there was a slight speedup (this was proposed as *memory-tuned Quicksort* by LaMarca and Ladner [28]).
- After partitioning, the pivot is moved to its correct position and not included in the recursive calls (not applied in `std::sort`).
- We always recurse on the smaller part of the partition first, thus, ensuring that the size of the recursion stack is bounded by $\log_2(n)$; see, e. g., [10, Problem 7-4] (also not applied in `std::sort`).
- The basic version of Quicksort uses a random or fixed element as pivot. A slight improvement is to choose the pivot as the median of three elements: typically, the first, the middle, and the last. This is applied in `std::sort` and many other Quicksort implementations.

Sedgewick [37] has already remarked that choosing the pivots from an even larger sample does not provide a significant increase in speed. In view of the experiments with skewed pivots [22], this is no surprise. For BlockQuicksort, a pivot closer to the median turns out to be slightly beneficial (Figure 3 in Section 4). Thus, it makes sense to invest more time to find a better pivot element since this also reduces the chance to have a really bad runtime. Martínez and Roura [32] show that the number of comparisons incurred by Quicksort is minimal if the pivot element is selected as median of $\Theta(\sqrt{n})$ elements. Another variant is to choose the pivot as median of three (resp., five) elements that themselves are medians of three (resp., five) elements. We implemented all of these variants for our experiments; see Section 4.

Quickselect. The *selection* problem is as follows: given an array of elements of some totally ordered universe and some integer k, find the kth smallest element in the sorted order of the array. Hoare's Quickselect [20] is the most common algorithm to solve this problem. It works almost like Quicksort: first, it chooses some pivot element (in the original version without pivot sampling, but usually implemented with median-of-three), performs the partitioning, and, finally, recurses on the side where the kth element lies. Thus, the difference to Quicksort is that the recursion is always only on one side of the array.

Like Quicksort, Quickselect also has a worst-case runtime of $\Theta(n^2)$; but its average complexity is $O(n)$. A way of improving Quickselect is to use adaptive sampling [30, 31]. Here, the idea is that if an element far off the median is searched for, then it might be beneficial not to choose the median of some sample as pivot but rather as an element more to one side. The Floyd-Rivest algorithm [15, 16] (see also [25]) applies this strategy to finding the median with only $\frac{3}{2}n + o(n)$ comparisons, on average (where the leading term is optimal). However, this algorithm uses two pivots, which is not suited for our block partitioner. Nevertheless, we give an implementation of Quickselect that, in practice, reaches a similar low number of comparisons, but without any theoretical proof.

Our main contribution is the block partitioner, which we describe in the next section.

3 BLOCK PARTITIONING

The idea of block partitioning is quite simple. Recall how Hoare's original partition procedure works (Algorithm 2): two pointers start at the leftmost and rightmost elements of the array and move toward the middle. In every step, the current element is compared to the pivot (Line 3 and 4). If some element on the right side is less than or equal to the pivot (resp., an element on the left side is greater than or equal to it), the respective pointer stops and the two elements found this way are swapped (Line 5). Then, the pointers continue moving toward the middle.

ALGORITHM 2: Hoare's Partitioning

```
1: procedure PARTITION(A[ℓ, ..., r], pivot)
2:     while ℓ < r do
3:         while A[ℓ] < pivot do ℓ++ end while
4:         while A[r] > pivot do r−− end while
5:         if ℓ < r then swap(A[ℓ], A[r]); ℓ++; r−− end if
6:     end while
7:     return ℓ
8: end procedure
```

The idea of BlockQuicksort (Algorithm 3) is to separate Lines 3 and 4 of Algorithm 2 from Line 5: fix some block size B; we introduce two buffer offsets$_L[0, \ldots, B-1]$ and offsets$_R[0, \ldots, B-1]$

for storing pointers to elements (offsets_L will store pointers to elements on the left side of the array that are greater than or equal to the pivot element – likewise, offsets_R for the right side). The main loop of Algorithm 3 consists of two stages: the scanning phase (Lines 5–18) and the rearranging phase (Lines 19–26).

ALGORITHM 3: Block Partitioning

```
1:  procedure BlockPartition(A[ℓ, ..., r], pivot)
2:      integer offsets_L[0, ..., B − 1], offsets_R[0, ..., B − 1]
3:      integer start_L, start_R, num_L, num_R ← 0
4:      while r − ℓ + 1 > 2B do                                   ▷ start main loop
5:          if num_L = 0 then                                     ▷ if left buffer is empty, refill it
6:              start_L ← 0
7:              for i = 0, ..., B − 1 do
8:                  offsets_L[num_L] ← i
9:                  num_L += (pivot ≥ A[ℓ + i])                   ▷ scanning phase for left side
10:             end for
11:         end if
12:         if num_R = 0 then                                     ▷ if right buffer is empty, refill it
13:             start_R ← 0
14:             for i = 0, ..., B − 1 do
15:                 offsets_R[num_R] ← i
16:                 num_R += (pivot ≤ A[r − i])                   ▷ scanning phase for right side
17:             end for
18:         end if
19:         integer num = min(num_L, num_R)
20:         for j = 0, ..., num − 1 do
21:             swap(A[ℓ + offsets_L[start_L + j]], A[r − offsets_R[start_R + j]])   ▷ rearranging phase
22:         end for
23:         num_L, num_R −= num;start_L, start_R += num
24:         if (num_L = 0) then ℓ += B end if
25:         if (num_R = 0) then r −= B end if
26:     end while                                                 ▷ end main loop
27:     scan and rearrange remaining elements
28: end procedure
```

Like for the classical Hoare partition, we also start with two pointers (or indices, as in the pseudocode) to the leftmost and rightmost element of the array. First, the scanning phase takes place: the buffers that are empty are refilled. In order to do so, we move the respective pointer toward the middle and compare each element with the pivot. However, instead of stopping at the first element that should be swapped, only a pointer to that element is stored in the respective buffer (Lines 8 and 9, resp., 15 and 16 – actually, the pointer is always stored but, depending on the outcome of the comparison, a counter holding the number of pointers in the buffer is increased or not) and the pointer continues moving toward the middle. After an entire block of B elements has been scanned (either on both sides of the array or only on one side), the rearranging phase begins: it starts with the first positions of the two buffers and swaps the data elements that they point to (Line 21); then, it continues until one of the buffers contains no more pointers to elements that should be swapped. Now, the scanning phase is restarted and the buffer that has run empty is filled again.

The algorithm continues this way until fewer elements than two times the block size remain. Now, the simplest variant is to switch to the usual Hoare partition method for the remaining

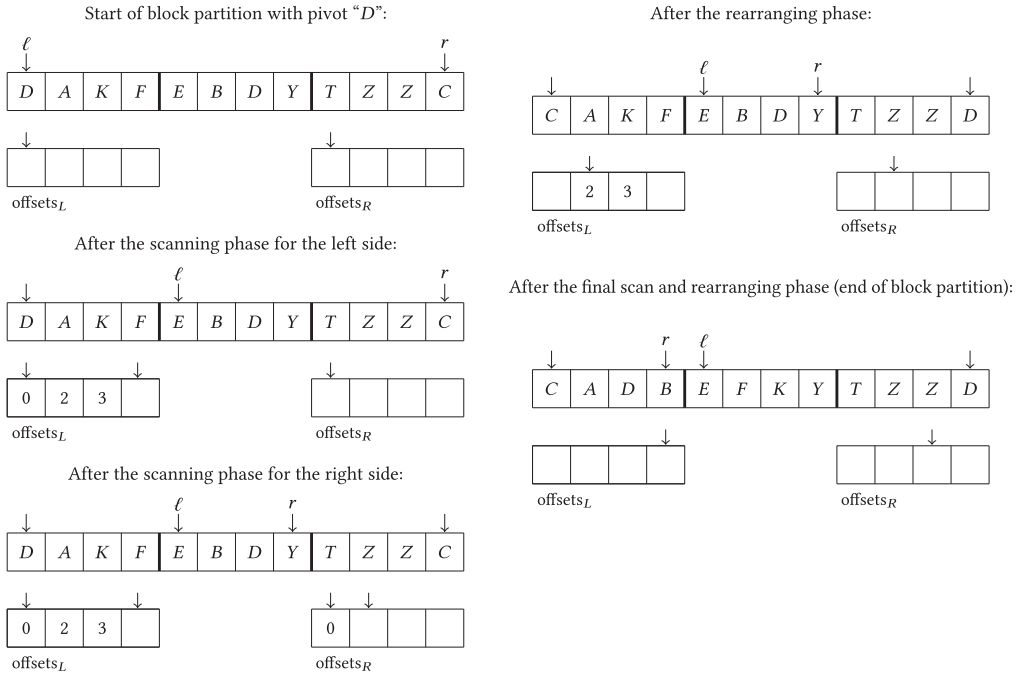

Fig. 1. Example of block partitioning with block size $B = 4$ and pivot "D."

elements (in the experiments with suffix `Hoare finish`). But, we also can continue with the idea of block partitioning: the algorithm scans the remaining elements as one or two final blocks (of smaller size) and performs a last rearranging phase. After that, one buffer might be empty but the other one might not be. With one run through the non-empty buffer, all of the respective elements can be moved to the left, respectively, right (similar to what is done in the Lomuto partitioning method but without performing actual comparisons). We do not present the details for this final rearranging here because it gets somewhat tedious and it neither provides a lot of insight into the algorithm nor is it necessary to prove our result on branch mispredictions. For an example of block partitioning with an array of 12 elements, see Figure 1.

3.1 Analysis

If the input consists of random permutations (all data elements are different), the average numbers of comparisons and swaps are the same as for the usual Quicksort with median-of-three. This is because both Hoare's partitioner and the block partitioner preserve randomness of the array.

The number of scanned elements (total number of elements loaded to the registers) is increased by two times the number of swaps because, for every swap, the data elements have to be loaded again. However, the idea is that, owing to the small block size, the data elements still remain in the L1 cache when being swapped; thus, the additional scan has no negative effect on runtime. In Section 4, we see that for larger data types and from a certain threshold on, an increasing size of the blocks has a negative effect on runtime. Therefore, the block size should not be chosen too large: we propose $B = 128$ and fix this constant throughout (thus, already for inputs of moderate size, the buffers also do not require much more space than the stack for Quicksort).

Branch Mispredictions. The next theorem is our main theoretical result. For simplicity, we assume here that BlockQuicksort is implemented without the worst-case-stopper Heapsort (i.e., there is

no limit on the recursion depth). Since there is only a low probability that a high recursion depth is reached while the array is still large, this assumption is not a real restriction. We analyze a static branch predictor: there is a misprediction every time a loop is left and a misprediction every time the *if* branch of an *if* statement is not taken.

THEOREM 3.1. *Let $C(n)$ be the average number of comparisons of Quicksort with a constant size pivot sample. Then, BlockQuicksort (without limit to the recursion depth and with the same pivot selection method) with block size B induces at most $\frac{6}{B} \cdot C(n) + O(n)$ branch mispredictions, on average. In particular, BlockQuicksort with median-of-three induces less than $\frac{8}{B} n \log n + O(n)$ branch mispredictions, on average.*

Theorem 3.1 shows that when choosing the block size sufficiently large, the $n \log n$-term becomes very small. In particular, for real-world input sizes, the linear term dominates the total number of branch misses by far – see also Figure 6. Theorem 3.1 holds also with samples of non-constant size for pivot selection as long as the sample size does not grow too fast. Moreover, the constant 6 in Theorem 3.1 can be replaced by 4 when implementing Lines 19, 24, and 25 of Algorithm 3 with conditional moves.

PROOF. First, we show that every execution of the block partitioner Algorithm 3 on an array of length n induces at most $\frac{6}{B} n + c$ branch mispredictions for some constant c. In order to do so, we need to look only at the main loop (Lines 4–27) of Algorithm 3 because the final scanning and rearranging phases consider only a constant (at most $2B$) number of elements. Inside the main loop, there are three *for* loops (starting Lines 7, 14, and 20), four *if* statements (starting Lines 5, 12, 24, and 25) and the min calculation (whose straightforward implementation is an *if* statement – Line 19). We know that in every execution of the main loop at least one of the conditions of the *if* statements in Lines 5 and 12 is true because, in every rearranging phase, at least one buffer runs empty. The same holds for the two *if* statements in Lines 24 and 25. Therefore, we obtain at most two branch mispredictions for the *if*s, three for the *for* loops and one for the min in every execution of the main loop.

In every execution of the main loop, there are at least B comparisons of elements with the pivot. Thus, the number of branch misses in the main loop is at most $\frac{6}{B}$ times the number of comparisons. Hence, for every input permutation, the total number of branch mispredictions of BlockQuicksort is at most $\frac{6}{B} \cdot$ #comparisons + $(c + c') \cdot$ #calls to partition + $O(n)$, where c' is the number of branch mispredictions of one execution of the main loop of Quicksort (including pivot selection, which needs only a constant number of instructions) and the $O(n)$ term comes from the final Insertionsort. The number of calls to partition is bounded by n because each element can be chosen as a pivot only once (since the pivots are not contained in the arrays for the recursive calls). Thus, by taking the average over all input permutations, the first statement follows.

The second statement follows because Quicksort with median-of-three incurs $1.18n \log n + O(n)$ comparisons, on average [36]. □

Remark 3.2. For a static branch predictor, the $O(n)$-term in Theorem 3.1 can be bounded by $3.125n$ by taking a closer look at the final rearranging phase.

We give a rough heuristic estimate: we assume that the average length of arrays on which Insertionsort is called is at least 8 (this is a fair approximation since we switch to Insertionsort as soon as the array size is less than 17). When executing Insertionsort, there is one branch miss for each element (when exiting the loop for finding the position) plus one for each call to Insertionsort (when exiting the loop over all elements to insert). Furthermore, there are at most two branch misses in the main Quicksort loop of our implementation for every call to Insertionsort (for the check whether to continue with Quicksort and for a possible switch to Heapsort). Hence, we have approximately $\frac{11}{8} n$ branch misses due to Insertionsort.

It remains to count the constant number of branch mispredictions incurred during each call of partitioning: there occurs one branch miss when exiting the main loop of block partition. After that, there is one more scan and rearranging phase with a smaller block size. This leads to at most 7 branch mispredictions (one extra because there is an additional case that both buffers are empty). The final rearranging incurs at most three branch misses. Selecting the pivot as median-of-three induces no branch misses since all conditional statements can be done via conditional moves. Finally, there is at most one branch miss in the Quicksort main loop following every call to partition (for the check whether to recurse on the left or right part first). This sums up to at most 14 additional branch misses per call to partition. Because the average size of arrays treated by Insertionsort is at least 8, the number of calls to partition is less than $n/8$.

Thus, in total, the $O(n)$-term in Theorem 3.1 consists of at most $\frac{11}{8}n + \frac{14}{8}n = 3.125n$ branch mispredictions.

Analysis of BlockQuickselect. Clearly, the block partitioner can be used for Quickselect as well – the resulting algorithm is called *BlockQuickselect*. The number of comparisons and swaps of BlockQuickselect is the same as for regular Quickselect – just like in the case of Quicksort. Also, for the number of scans, we are in the same situation as for Quicksort. For the number of branch misses with a static predictor, we can prove an analog upper bound:

Corollary 3.3. *Let $C(n)$ be the average number of comparisons of Quickselect with constant size pivot sample to find the median of n elements. Then, BlockQuickselect (without limit to the recursion depth and with the same pivot selection method) with block size B induces at most $\frac{6}{B} \cdot C(n) + O(\log n)$ branch mispredictions, on average. In particular, BlockQuickselect with median-of-three induces less than $\frac{16.5}{B} n \log n + o(n)$ branch mispredictions, on average.*

Proof. The proof is the same as for Theorem 3.1. Note that, here, the expected number of recursive calls to Quickselect is in $O(\log(n))$ – no matter how the pivot is selected. The second statement is because Quickselect with median-of-three uses at most $2.75n + o(n)$ comparisons, on average [24]. □

3.2 Further Tuning of Block Partitioning

We propose and have implemented further tunings for our block partitioner:

(1) Loop unrolling: Since the block size is a power of two, the loops of the scanning phase can be unrolled four or even eight times without causing additional overhead.
(2) Cyclic permutations instead of swaps: We replace

```
for j = 0, ..., num − 1 do
    swap(A[ℓ + offsets_L[start_L + j]], A[r − offsets_R[start_R + j]])
end for
```

by the following code, which does not perform exactly the same data movements, but still, in the end, all elements less than the pivot are on the left and all elements greater are on the right:

```
temp ← A[ℓ + offsets_L[start_L]]
A[ℓ + offsets_L[start_L]] ← A[r − offsets_R[start_R]]
for j = 1, ..., num − 1 do
    A[r − offsets_R[start_R + j − 1]] ← A[ℓ + offsets_L[start_L + j]]
    A[ℓ + offsets_L[start_L + j]] ← A[r − offsets_R[start_R + j]]
end for
A[r − offsets_R[start_R + num − 1]] ←temp
```

Note that this is also a standard improvement for partitioning – see, for example, [3].

In the following, we always assume these two improvements since they are of a very basic nature (plus one more small change in the final rearranging phase). We call the variant without them `block partition simple`.

The next improvement is a slight change of the algorithm. In our experiments, we noticed that for small arrays with many duplicates, the recursion depth becomes often higher than the threshold for switching to Heapsort. A way to circumvent this is an additional check for duplicates equal to the pivot if one of the following two conditions applies:

- the pivot occurs twice in the sample for pivot selection (in the case of median-of-three) or
- the partitioning is very unbalanced for an array of small size.

The check for duplicates takes place after the partitioning is completed. Only the larger half of the array is searched for elements equal to the pivot. This check works similar to Lomuto's partitioner (we used Katajainen's implementation [23]): starting from the position of the pivot, the respective half of the array is scanned for elements equal to the pivot (this can be done by one *less than* comparison since elements are already known to be greater than or equal to — resp., less than or equal to — the pivot). Elements that are equal to the pivot are moved to the side of the pivot. The scan continues as long as at least every fourth element is equal to the pivot (instead, every fourth one could take any other ratio – this guarantees that the check stops soon if there are only a few duplicates).

After this check, all elements that are identified as being equal to the pivot remain in the middle of the array (between the elements larger and the elements smaller than the pivot); thus, they can be excluded from further recursive calls. We denote this version with the suffix `duplicate check` (dc).

4 EXPERIMENTS

We ran thorough experiments with implementations in C++ on different machines with different types of data and different kinds of input sequences. If not specified explicitly, the experiments are run on an Intel Core i5-2500K CPU (3.30GHz, 4 cores, 32KB L1 instruction and data cache, 256KB L2 cache per core and 6MB L3 shared cache) with 16GB RAM and operating system Ubuntu Linux 64b version 14.04.4. We used GNU's `g++` (4.8.4), optimized with flags `-O3 -march=native`.

For time measurements, we used `std::chrono::high_resolution_clock`, for generating random inputs, the Mersenne Twister pseudo-random generator `std::mt19937`. All time measurements were repeated with the same 20 deterministically chosen seeds – the displayed numbers are averages of these 20 runs. For each time measurement, at least 128MB of data were sorted. If the array size is smaller, then for this time measurement several arrays have been sorted and the total elapsed time measured. Our runtime plots all display the actual time divided by the number of elements to sort on the `y` axis.

We performed our runtime experiments with three different data types:

- `int`: signed 32b integers.
- `Vector`: 10-dimensional array of 64b floating-point numbers (`double`). The order is defined via the Euclidean norm – for every comparison, the sums of the squares of the components are computed and then compared.
- `Record`: 21-dimensional array of 32b integers. Only the first component is compared.

The code of our implementation of BlockQuicksort as well as the other algorithms and our runtime experiments is available at https://github.com/weissan/BlockQuicksort.

Different Block Sizes. Figure 2 shows experimental results on random permutations for different data types and block sizes ranging from 4 up to 2^{24}. We see that for integers only at the end,

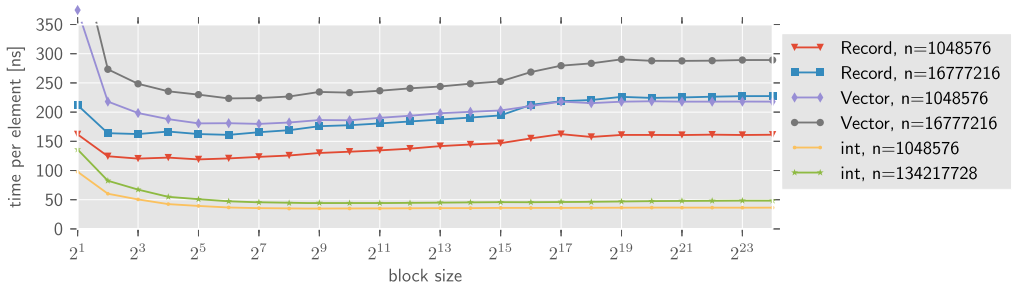

Fig. 2. Different block sizes for random permutations.

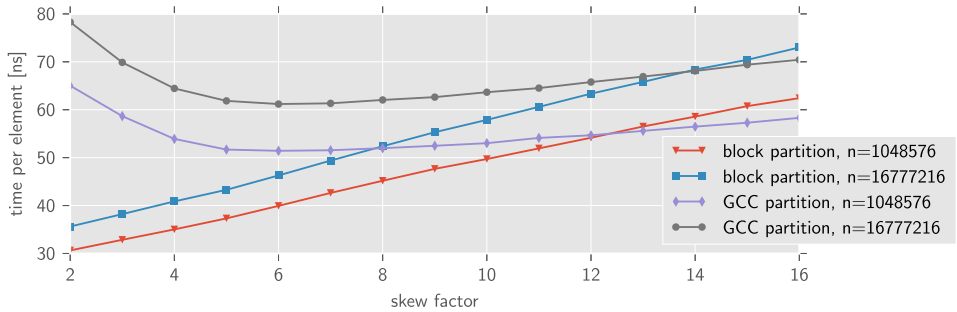

Fig. 3. Sorting random permutations of 32b integers with skewed pivot. A skew factor k means that $\lfloor \frac{n}{k} \rfloor$th element is chosen as the pivot of an array of length n.

there is a slight negative effect when increasing the block size. Presumably, this is because up to a block size of 2^{19}, two blocks still fit entirely into the L3 cache of the CPU. On the other hand, for `Vector`, a block size of 64 and for `Record` a block size of 8 seem to be optimal – with a considerably increasing runtime for larger block sizes.

As a compromise, we chose to fix the block size to 128 elements for all further experiments. An alternative approach would be to choose a fixed number of bytes for one block and adapt the block size according to the size of the data elements.

Skewed Pivot Experiments. We repeated the experiments of Kaligosi and Sanders with skewed pivot [22] for both the usual Hoare partitioner (`std::__unguarded_partition`, from the GCC implementation of `std::sort`) and our block partitioner. For both partitioners, we used our tuned Quicksort loop. The results can be seen in Figure 3: classic Quicksort benefits from a skewed pivot, whereas BlockQuicksort works best with the exact median. Therefore, for BlockQuicksort, it makes sense to invest more effort into finding a good pivot.

Different Pivot Selection Methods. We implemented several strategies for pivot selection:

- median-of-three, median-of-five, median-of-twenty-three;
- median-of-three-medians-of-three, median-of-three-medians-of-five, median-of-five-medians-of-five: first, calculate three (resp., five) times the median of three (resp., five) elements, then take the pivot as the median of these three (resp., five) medians;
- median-of-$\sqrt{n}$.

All pivot selection strategies switch to median-of-three for small arrays. Moreover, the median-of-$\sqrt{n}$ variant switches to median-of-five-medians-of-five for arrays of length below 20,000 (for smaller n even the number of comparisons was better with median-of-five-medians-of-five). The

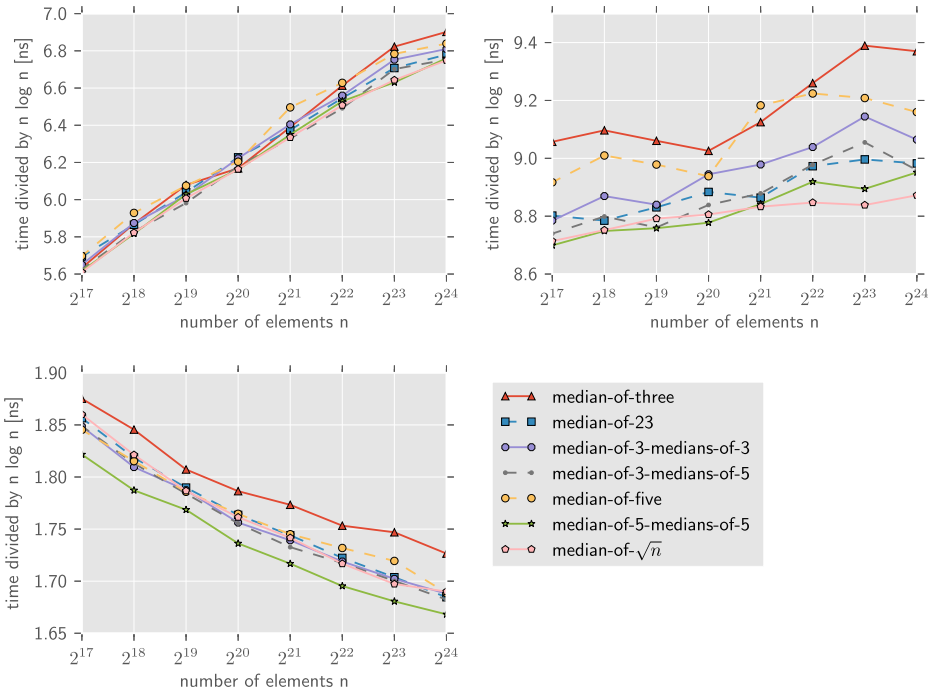

Fig. 4. Different pivot selection strategies with random permutation. *Upper left*: `Record`; *upper right*: `Vector`; *lower left*: `int`. Be aware that the y axis here displays the time divided by $n \log n$.

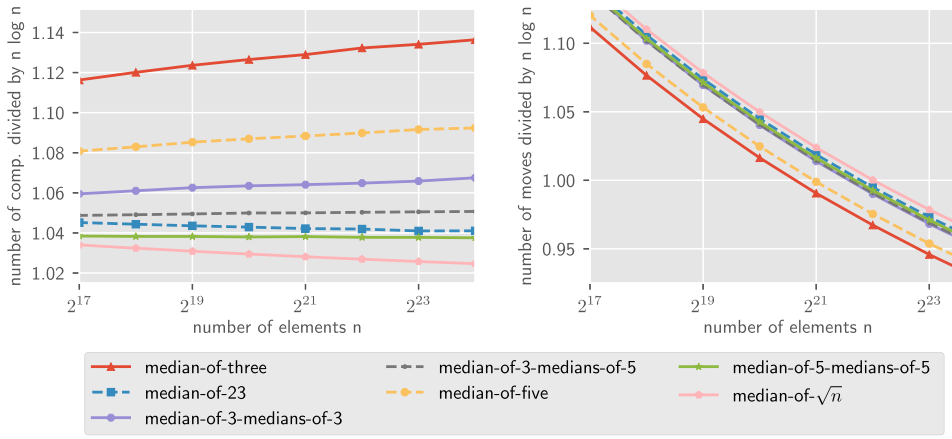

Fig. 5. Number of comparisons (*left*) and moves (*right*) for different pivot selection strategies with random permutation. Here, the y axis displays the number of comparisons (resp., moves) divided by $n \log n$.

medians of larger samples are computed with `std::nth_element`. Despite the results on skewed pivots in Figure 3, there was no big difference between the different pivot selection strategies, as can be seen in Figure 4. As expected, median-of-three was always the slowest for larger arrays. Median-of-five-medians-of-five was the fastest for `int` and median-of-$\sqrt{n}$ was the fastest for `Vector`. In Figure 5, we count the number of comparisons and moves incurred by BlockQuicksort with different pivot selection strategies. In terms of comparisons, except median-of-three, all of

the strategies are very close to each other and even the difference between median-of-three and median-of-$\sqrt{n}$ is only roughly 10%. In particular, we see that, already, median-of-five gives relatively good pivots. The difference for the moves is even less than for the number of comparisons. Moreover, median-of-three, the one with the most comparisons, uses the least number of moves. This explains the small difference between the runtimes of the different pivot selection strategies. Since for the `Record` type, owing to the large data types but only small comparison keys, the moves contribute a rather large portion to the overall runtime, we can expect a rather small difference between the runtimes – which agrees with the observation in Figure 4.

4.1 Comparison with Other Sorting Algorithms

We compare variants of BlockQuicksort with the GCC implementation of `std::sort`[3] (which is known to be one of the most efficient Quicksort implementations – e.g., see [8]), Yaroslavskiy's dual-pivot Quicksort [41] (we converted the Java code of [41] to C++) and an implementation of Super Scalar Sample Sort [35] by Hübschle-Schneider[4]. For random permutations and random values modulo $\sqrt{n}$, we also test Tuned Quicksort [23], three-pivot Quicksort implemented by Aumüller and Bingmann[5] [5] (which is based on [27]), and a Quicksort implementation using our optimized Quicksort main loop together with the GCC partition from `std::sort` (in order to see the effects from improving only the partitioning method). For simplicity, we omit these algorithms for other types of inputs.

Branch Mispredictions. We experimentally determined the number of branch mispredictions of BlockQuicksort and the other algorithms with the *chachegrind* branch prediction profiler, which is part of the profiling tool *valgrind*[6]. The results of these experiments on random `int` data can be seen in Figure 6; the `y` -axis shows the number of branch mispredictions divided by the array size. We display only the median-of-three variant of BlockQuicksort since all variants are very much alike. We also added plots of BlockQuicksort and Tuned Quicksort, skipping final Insertionsort (i.e., the arrays remain partially unsorted).

We see that both `std::sort` and Yaroslavskiy's dual-pivot Quicksort incur $\Theta(n \log n)$ branch mispredictions. The up and down for Super Scalar Sample Sort presumably is because of the variation in the size of the arrays where the base case sorting algorithm `std::sort` is applied. For BlockQuicksort, there is an almost non-visible $n \log n$ term for the number of branch mispredictions. We computed an approximation of $0.02n \log n + 1.75n$ branch mispredictions. Thus, the actual number of branch mispredictions is still better than our bounds in Theorem 3.1 and Remark 3.2 (which are $0.0625n \log n + 3.125n$ for a block size of 128). There are two factors that contribute to this discrepancy: first, our rough worst-case estimate of the branch misses during partitioning and, second, the actual branch predictor of a modern CPU might be better than a static branch predictor. Also, note that more than half of the branch mispredictions are incurred by

[3]For the source code, see https://gcc.gnu.org/onlinedocs/gcc-4.7.2/libstdc++/api/a01462_source.html. Note that in newer versions of GCC, the implementation is slightly different: the old version uses the first, middle, and last elements as samples for pivot selection, whereas the new version uses the *second*, middle, and last elements. For decreasingly sorted arrays, the newer version works far better; for random permutations and increasingly sorted arrays, the old one is better. We used the old version for our experiments. The new version is included in the plots of Figure 11; this reveals a enormous difference between the two versions for particular inputs and underlines the importance of proper pivot selection.

[4]https://github.com/lorenzhs/ssssort/.

[5]http://eiche.theoinf.tu-ilmenau.de/Quicksort-experiments/.

[6]For more information on valgrind, see http://valgrind.org/. To perform the measurements, we used the same Python script as in [13, 23], which first measures the number of branch mispredictions of the whole program, including generation of test cases, and then, in a second run, measures the number of branch mispredictions incurred by the generation of test cases.

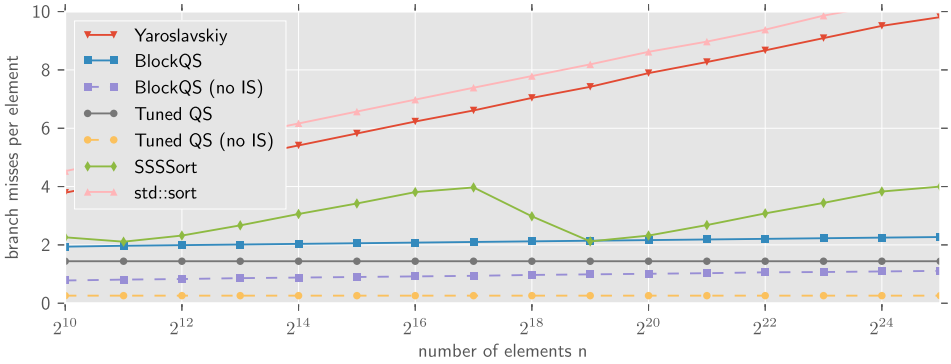

Fig. 6. Number of branch mispredictions.

Table 1. Instruction Count, Branch and Cache Misses When Sorting Random `int` Permutations of Size $16777216 = 2^{24}$

Algorithm	Branches taken	Branch misses	Instructions	L1 refs	L3 refs	L3 misses
`std::sort`	37.81	10.23	174.82	51.96	1.05	0.41
SSSSort	16.2	3.87	197.06	68.47	0.82	0.5
Yaroslavskiy	52.92	9.51	218.42	59.82	0.79	0.27
BlockQS (mo-$\sqrt{n}$, dc)	20.55	2.39	322.08	89.9	0.77	0.27
BlockQS (mo5-mo5)	20.12	2.31	321.49	88.63	0.78	0.28
BlockQS	20.51	2.25	337.27	92.45	0.88	0.3
BlockQS (no IS)	15.38	1.09	309.85	84.66	0.88	0.3
Tuned QS	29.66	1.44	461.88	105.43	1.23	0.39
Tuned QS (no IS)	24.53	0.26	434.53	97.65	1.22	0.39

All displayed numbers are divided by the number of elements.

Insertionsort; less than one half are incurred by the actual block partitioning and Quicksort main loop. Moreover, the number of branch mispredictions incurred by Insertionsort almost matches our estimate in Remark 3.2.

Finally, Figure 6 shows that Tuned Quicksort by Katajainen et al. is still more efficient with respect to branch mispredictions (only $O(n)$). This is no surprise since it does not need any checks as to whether buffers are empty and so forth. Moreover, we see that over 80% of the branch misses of Tuned Quicksort come from the final Insertionsort.

More Profiling Statistics. Table 1 shows the number of branches taken/branches mispredicted as well as the instruction count and cache misses. Although `std::sort` has a much lower instruction count than the other algorithms, it induces most branch misses and (except Tuned Quicksort) most L1 cache misses (= L3 refs since no L2 cache is simulated). BlockQuicksort does not only have a low number of branch mispredictions but also a good cache behavior. One reason for this is that Insertionsort is applied during the recursion and not at the very end.

Runtime Experiments. In Figure 7, we present runtimes on random `int` permutations of different BlockQuicksort variants and the other algorithms, including Katajainen's Tuned Quicksort and Aumüller and Bingmann's three-pivot Quicksort. The optimized BlockQuicksort variants need around 45ns per element when sorting 2^{28} elements, whereas `std::sort` needs 85ns per element. Thus, there is a speed increase of 88% (i.e., the number of elements sorted per second is increased

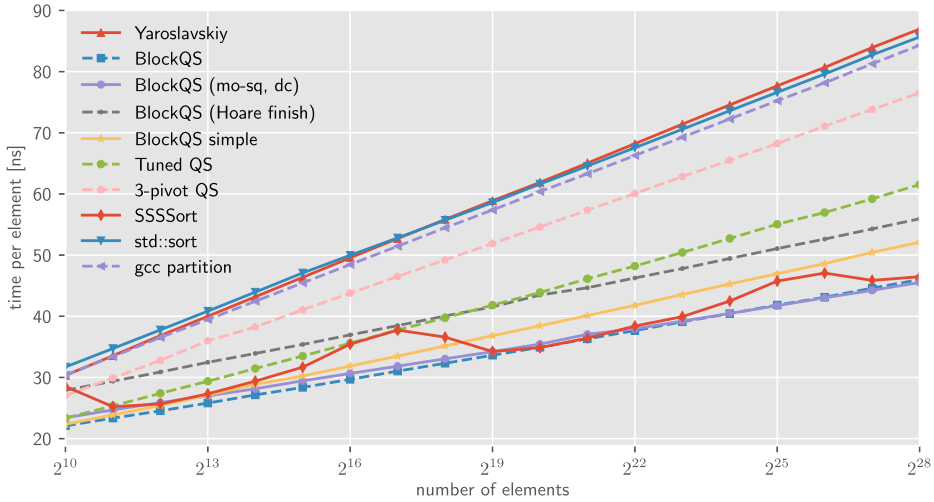

Fig. 7. Random permutations of `int`.

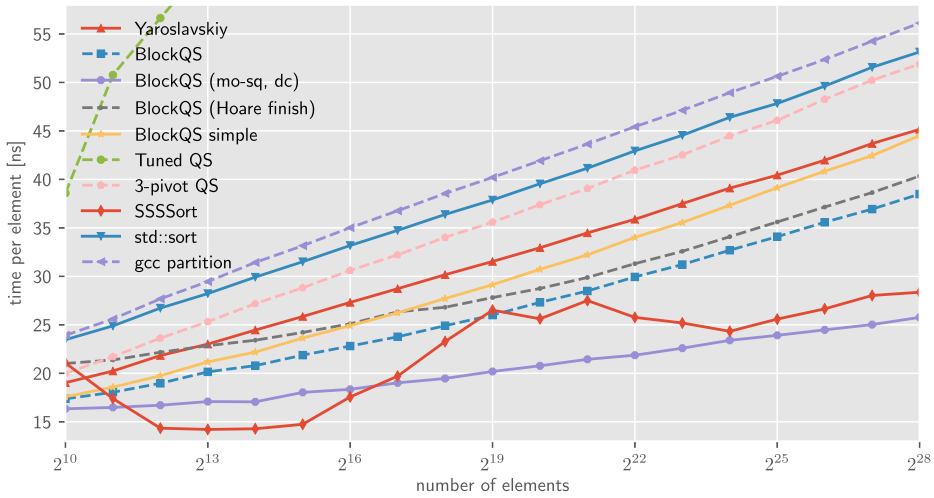

Fig. 8. Random `int` values between 0 and $\sqrt{n}$.

by 88%). The optimized Quicksort with GCC partition (which, except for the partitioning, is the same as BlockQuicksort with median-of-three) is only slightly faster than `std::sort`. Thus, almost the whole speedup of BlockQuicksort is due to the improved partitioning algorithm. The same algorithms are displayed in Figure 8 for sorting random `ints` between 0 and $\sqrt{n}$. Here, we observe that Tuned Quicksort is much worse than all the other algorithms (already, for $n = 2^{12}$, it moves outside the displayed range). All variants of BlockQuicksort are faster than `std::sort`; the `duplicate check` (dc) version is almost twice as fast. Here, the optimized Quicksort with GCC partition is even slower than `std::sort`. One reason for this might be the different choices of pivot elements.

Figure 9 presents experiments with data containing many duplicates and having specific structures, thus, perhaps coming closer to "real-world" inputs (although it is not clear what that means). Since here Tuned Quicksort and three-pivot Quicksort are much slower than all of the other

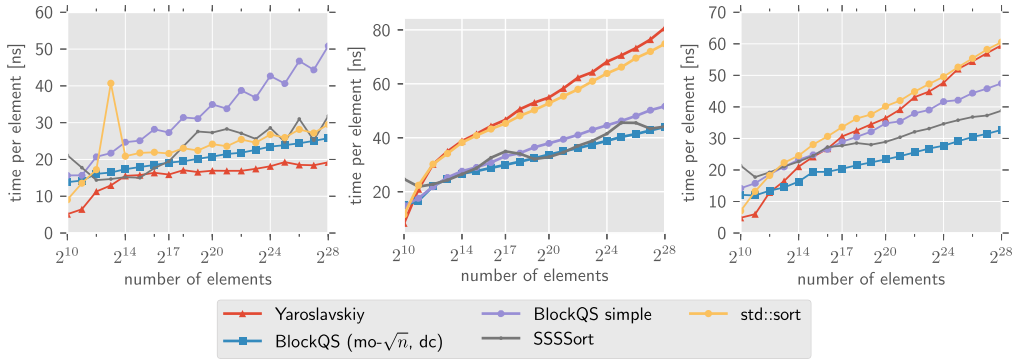

Fig. 9. Arrays A of `int` with duplicates. *Left*: $A[i] = i \mod \lfloor \sqrt{n} \rfloor$; *middle*: $A[i] = i^2 + n/2 \mod n$; *right*: $A[i] = i^8 + n/2 \mod n$. Since n is always a power of two, the value $n/2$ occurs approximately $n^{7/8}$ times in the last case.

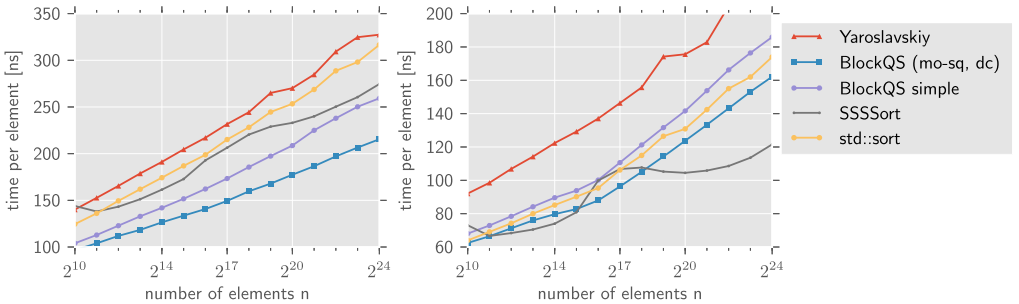

Fig. 10. Random permutations. *Left:* `Vector`; *right:* `Record`.

algorithms, we exclude these two algorithms from the plots. The array for the left plot contains long already sorted runs. This is most likely the reason that `std::sort` and Yaroslavskiy's dual-pivot Quicksort have similar runtimes to BlockQuicksort (for sorted sequences, the conditional branches can be easily predicted as to what explains the fast runtime). The arrays for the middle and right plot start with sorted runs and become increasingly more erratic; the array for the right one also contains a extremely high number of duplicates. Here, the advantage of BlockQuicksort – avoiding conditional branches – can be observed again. In all three plots, we see that the check for duplicates (dc) means a considerable improvement.

In Figure 10, we show the results of selected algorithms for random permutations of `Vector` and `Record`. We conjecture that the good results of Super Scalar Sample Sort on `Record`s are because of its better cache behavior (since `Record` has large data elements with very cheap comparisons).

In Figure 11, we see the runtimes for special permutations: already sorted, reversed, and a cyclic shift by $n/2$. Because of the strange behavior of `std::sort`, we also included the new GCC implementation of `std::sort` (GCC version 4.8.4) marked as `std::sort` (new). The very small difference in the implementation of choosing the second instead of the first element as part of the sample for pivot selection makes an enormous difference when sorting special permutations such as decreasingly sorted arrays. This shows how important not only the size of the pivot sample is but also proper selection. In the other benchmarks, both implementations were relatively close; thus, we do not show both. If the array is already sorted, no swaps have to be performed for partitioning. Therefore, all of the respective comparisons are easily predictable, and it is no surprise that both

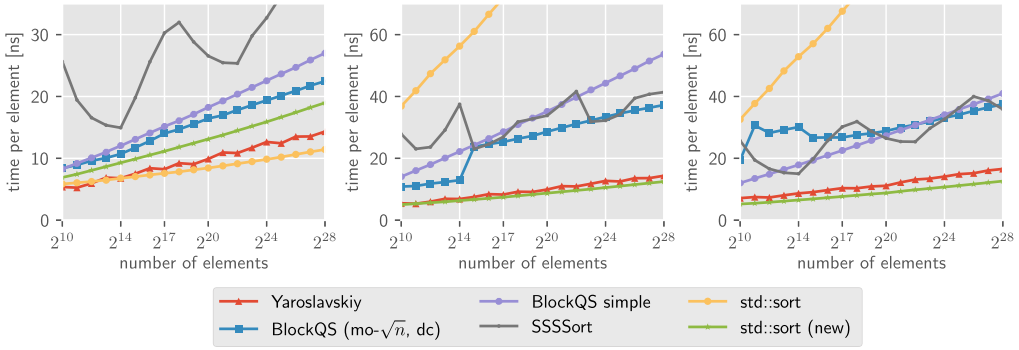

Fig. 11. Permutations of `int`. *Left*: sorted; *middle*: reversed; *right*: transposition – $A[i] = i + n/2 \mod n$.

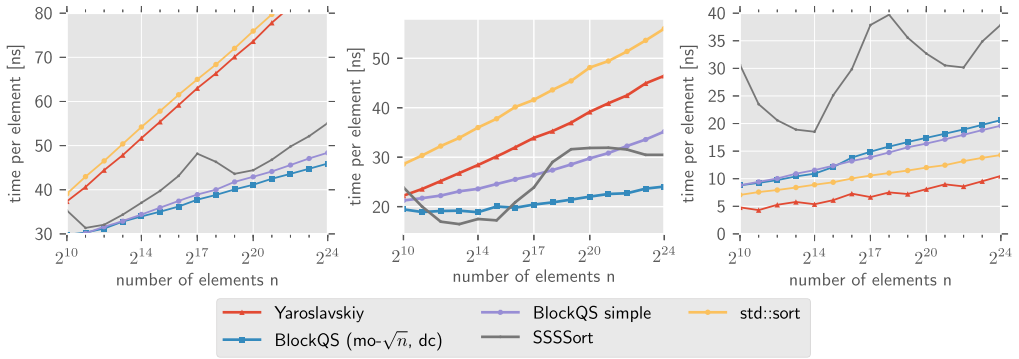

Fig. 12. Runtime experiments with `int` on a laptop with Intel Core i7-5500U CPU (2.40GHz, 2 cores, 32KB L1 instruction and data cache, 256KB L2 cache per core and 4MB L3 shared cache) with 8GB RAM and operating system Windows 10. We used Mingw g++ (5.3.0); optimized with flags `-O3 -march=native`.
Left: random permutation; *middle*: random values between 0 and $\sqrt{n}$; *right*: sorted.

versions of `std::sort` beat BlockQuicksort here (although BlockQuicksort also benefits from presorted inputs). In Figure 12, we conclude with some experiments conducted on another machine. The results show a similar behavior; only our improvements over the simple BlockQuicksort variant have less effect here.

4.2 Experiments on Quickselect

We also ran experiments with BlockQuickselect, comparing it to the GCC implementation `std::nth_element` of Quickselect. Because of the higher variance of the runtimes of Quickselect, all times are averages over measurements with the same 50 (instead of only 20) deterministically chosen seeds. Otherwise, the setting is exactly the same as for sorting.

In addition to the basic BlockQuickselect (with median-of-three) and `std::nth_element`, we compared BlockQuickselect with median-of-$\sqrt{n}$ and adaptive versions of both BlockQuickselect and classical Quickselect, where the pivot is chosen from a sample of size approximately $\sqrt{n}$. For the classical Quickselect, we use as partitioner `std::__unguarded_partition` from the GCC implementation of `std::sort`; otherwise, the implementation agrees with the adaptive version of BlockQuickselect.

In Figures 13 and 14, we see that on `int`s, BlockQuickselect is approximately twice as fast as Quickselect with the GCC partitioner. The non-adaptive version of BlockQuickselect is even faster

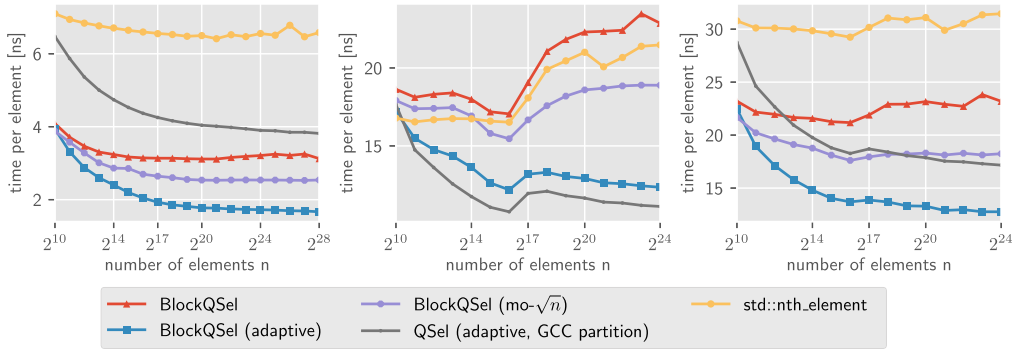

Fig. 13. Quickselect on random permutations. *Left:* `int`; *middle:* `Record`; *right:* `Vector`.

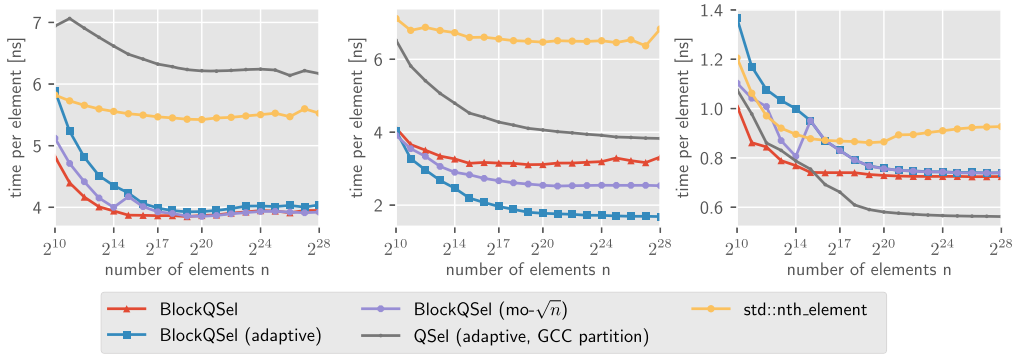

Fig. 14. Arrays A of `int`. *Left*: random 0-1 values; *middle*: Random values between 0 and $\sqrt{n}$; *right*: sorted.

than the adaptive version with classical partitioning. Also, on `Vectors`, we can see a significant speed-up by BlockQuickselect; only on `Records` does the GCC partitioner perform slightly better than the block partitioner. Except for 0-1 values, the adaptive versions perform considerably better than their non-adaptive counterparts.

5 CONCLUSIONS AND FUTURE RESEARCH

We have established an efficient in-place general-purpose sorting algorithm, which avoids branch predictions by converting results of comparisons to integers. In the experiments, we have seen that it is competitive on different kinds of data. Moreover, in several benchmarks it is almost twice as fast as `std::sort`. Future research might address the following issues.

- We used Insertionsort as recursion stopper, inducing a linear number of branch misses. Is there a more efficient recursion stopper that induces fewer branch mispredictions?
- More efficient usage of the buffers: In our implementation the buffers are not even filled halfway, on average. To use the space more efficiently, one could address the buffers cyclically and scan until one buffer is filled. By doing so, both buffers could be filled in the same loop, with the cost of introducing additional overhead, however.
- The final rearranging of the block partitioner is not optimal: For small arrays, similar problems with duplicates arise, as for Lomuto's partitioner.
- Pivot selection strategy: Though theoretically optimal, median-of-$\sqrt{n}$ pivot selection is not best in practice. Also, we want to emphasize that not only the sample size but also the

selection method is important (compare the different behaviors of the two versions of `std::sort` for sorted and reversed permutations). It might even be beneficial to use a fast pseudo-random number generator (e.g., a linear congruence generator) for selecting samples for pivot selection.

- Parallel versions: The block structure is very well suited for parallelism.
  While preparing this article, a new in-place variant of Super Scalar Sample Sort appeared, called IPS^{4}o [6]. The idea behind it is to make Super Scalar Sample Sort in place using a constant size buffer for each bucket. Its main advantage is that it scales very well in a parallel implementation. Still, Axtmann et al. show that the single-threaded version of IPS^{4}o is approximately even 20% faster than BlockQuicksort [6, Figure 6]. On the other hand, the constant amount of space that it needs is considerably larger than of BlockQuicksort for any instance of practical size. Moreover, IPS^{4}o uses BlockQuicksort to sort small sub-arrays.
- Multi-pivot variants of BlockQuicksort might be interesting because of their better cache behavior. In particular, we suggest three pivots because it allows a balanced search tree for the position between the pivots. However, efficient multi-pivot variants are not trivial: our first attempt was much slower. Recently, in the student project [17], a dual pivot version of BlockQuicksort was studied (the code is also available in our repository on github); but their results were rather disappointing. The best dual-pivot variant took over 50% longer to sort 2^{28} random `ints`.

ACKNOWLEDGMENTS

Thanks to Jyrki Katajainen and Max Stenmark for allowing us to use their Python scripts for measuring branch mispredictions and cache misses and to Lorenz Hübschle-Schneider for his implementation of Super Scalar Sample Sort. We are also indebted to Jan Philipp Wächter for all his help with creating the plots, to Daniel Bahrdt for answering many C++ questions, and to Christoph Greulich for his help with the experiments.

REFERENCES

[1] 2011. ARMv8 Instruction Set Overview. Retrieved December 24, 2018 from https://www.element14.com/community/servlet/JiveServlet/previewBody/41836-102-1-229511/ARM.Reference_Manual.pdf Document number: PRD03-GENC-010197 15.0.

[2] 2016. Intel 64 and IA-32 Architecture Optimization Reference Manual. Retrieved December 24, 2018 from http://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf Order Number: 248966-032.

[3] D. Abhyankar and M. Ingle. 2011. Engineering of a Quicksort partitioning algorithm. *Journal of Global Research in Computer Science* 2, 2 (2011), 17–23.

[4] Martin Aumüller and Martin Dietzfelbinger. 2013. Optimal partitioning for dual pivot Quicksort (Extended abstract). In *Proceedings of Automata, Languages, and Programming - 40th International Colloquium (ICALP'13), Riga, Latvia, July 8-12, 2013, Part I, Lecture Notes in Computer Science*, Fedor V. Fomin, Rusins Freivalds, Marta Z. Kwiatkowska, and David Peleg (Eds.), Vol. 7965. Springer, Berlin, 33–44. DOI: https://doi.org/10.1007/978-3-642-39206-1_4

[5] Martin Aumüller, Martin Dietzfelbinger, and Pascal Klaue. 2016. How good is multi-pivot Quicksort? *ACM Trans. Algorithms* 13, 1 (2016), 8:1–8:47. DOI: https://doi.org/10.1145/2963102

[6] Michael Axtmann, Sascha Witt, Daniel Ferizovic, and Peter Sanders. 2017. In-place parallel super scalar Samplesort (IPSSSSo). In *25th Annual European Symposium on Algorithms (ESA'17), September 4-6, 2017, Vienna, Austria (LIPIcs)*, Kirk Pruhs and Christian Sohler (Eds.), Vol. 87. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 9:1–9:14. DOI: https://doi.org/10.4230/LIPIcs.ESA.2017.9

[7] Paul Biggar, Nicholas Nash, Kevin Williams, and David Gregg. 2008. An experimental study of sorting and branch prediction. *J. Exp. Algorithmics* 12 (2008), 1.8:1–39.

[8] Gerth Stølting Brodal, Rolf Fagerberg, and Kristoffer Vinther. 2008. Engineering a cache-oblivious sorting algorithm. *J. Exp. Algorithmics* 12 (2008), 2.2:1–23.

[9] Gerth Stølting Brodal and Gabriel Moruz. 2005. Tradeoffs between branch mispredictions and comparisons for sorting algorithms. In *WADS. Lecture Notes in Computer Science*, Vol. 3608. Springer, Berlin, 385–395.

[10] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2001. *Introduction to Algorithms* (2nd ed.). The MIT Press.

[11] Stefan Edelkamp and Armin Weiß. 2016. BlockQuicksort: Avoiding branch mispredictions in Quicksort. In *24th Annual European Symposium on Algorithms (ESA'16), August 22-24, 2016, Aarhus, Denmark (LIPIcs)*, Piotr Sankowski and Christos D. Zaroliagis (Eds.), Vol. 57. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 38:1–38:16. DOI: https://doi.org/10.4230/LIPIcs.ESA.2016.38

[12] Amr Elmasry and Jyrki Katajainen. 2012. Lean programs, branch mispredictions, and sorting. In *Proceedings of Fun with Algorithms (FUN'12). Lecture Notes in Computer Science*, Evangelos Kranakis, Danny Krizanc, and Flaminia L. Luccio (Eds.), Vol. 7288. Springer, Berlin, 119–130. DOI: https://doi.org/10.1007/978-3-642-30347-0_14

[13] Amr Elmasry, Jyrki Katajainen, and Max Stenmark. 2012. Branch mispredictions don't affect Mergesort. In *Proceedings of Experimental Algorithms - 11th International Symposium (SEA'12), Bordeaux, France, June 7-9, 2012. Lecture Notes in Computer Science*, Ralf Klasing (Ed.), Vol. 7276. Springer, Berlin, 160–171. DOI: https://doi.org/10.1007/978-3-642-30850-5_15

[14] Robert W. Floyd. 1964. Algorithm 245: Treesort 3. *Comm. ACM* 7, 12 (1964), 701.

[15] Robert W. Floyd and Ronald L. Rivest. 1975. The algorithm SELECT - for finding the ith smallest of n elements [M1] (Algorithm 489). *Commun. ACM* 18, 3 (1975), 173. DOI: https://doi.org/10.1145/360680.360694

[16] Robert W. Floyd and Ronald L. Rivest. 1975. Expected time bounds for selection. *Commun. ACM* 18, 3 (1975), 165–172. DOI: https://doi.org/10.1145/360680.360691

[17] Nikolaj Hass and Mikkel Angaju Rasmussen. 2016. Is Multi-Pivot BlockQuickSort viable? Unpublished student project supervised by Martin Aumüller.

[18] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture: A Quantitative Approach* (5th ed.). Morgan Kaufmann.

[19] Charles A. R. Hoare. 1961. Algorithm 64: Quicksort. *Commun. ACM* 4, 7 (1961), 321. DOI: https://doi.org/10.1145/366622.366644

[20] Charles A. R. Hoare. 1961. Algorithm 65: Find. *Commun. ACM* 4, 7 (1961), 321–322. DOI: https://doi.org/10.1145/366622.366647

[21] Charles A. R. Hoare. 1962. Quicksort. *Comput. J.* 5, 1 (1962), 10–16.

[22] Kanela Kaligosi and Peter Sanders. 2006. How branch mispredictions affect Quicksort. In *Proceedings of Algorithms - 14th Annual European Symposium (ESA'06)*, Zurich, Switzerland, September 11-13, 2006, *Lecture Notes in Computer Science*, Yossi Azar and Thomas Erlebach (Eds.), Vol. 4168. Springer, Berlin, 780–791. DOI: https://doi.org/10.1007/11841036_69

[23] Jyrki Katajainen. 2014. *Sorting Programs Executing Fewer Branches.* CPH STL Report 2263887503. Department of Computer Science, University of Copenhagen.

[24] Peter Kirschenhofer, Helmut Prodinger, and Conrado Martinez. 1997. Analysis of Hoare's FIND algorithm with median-of-three partition. *Random Struct. Algorithms* 10, 1-2 (1997), 143–156. DOI: https://doi.org/10.1002/(SICI)1098-2418(199701/03)10:1/2⟨143::AID-RSA7⟩3.0.CO;2-V

[25] Krzysztof C. Kiwiel. 2005. On Floyd and Rivest's SELECT algorithm. *Theor. Comput. Sci.* 347, 1-2 (2005), 214–238. DOI: https://doi.org/10.1016/j.tcs.2005.06.032

[26] Donald E. Knuth. 1998. *Sorting and Searching* (2nd ed.). The Art of Computer Programming, Vol. 3. Addison Wesley Longman.

[27] Shrinu Kushagra, Alejandro López-Ortiz, Aurick Qiao, and J. Ian Munro. 2014. Multi-pivot Quicksort: Theory and experiments. In *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX'14)*, Portland, Oregon, January 5, 2014, Catherine C. McGeoch and Ulrich Meyer (Eds.). SIAM, 47–60. DOI: https://doi.org/10.1137/1.9781611973198.6

[28] Anthony LaMarca and Richard E. Ladner. 1999. The influence of caches on the performance of sorting. *J. Algorithms* 31, 1 (1999), 66–104.

[29] Conrado Martínez, Markus E. Nebel, and Sebastian Wild. 2015. Analysis of branch misses in Quicksort. In *Workshop on Analytic Algorithmics and Combinatorics (ANALCO'15)*, San Diego, CA, January 4, 2015. 114–128.

[30] Conrado Martinez, Daniel Panario, and Alfredo Viola. 2004. Adaptive sampling for Quickselect. In *Proceedings of the 15th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA'04), New Orleans, Louisiana, January 11-14, 2004*, J. Ian Munro (Ed.). SIAM, 447–455. http://dl.acm.org/citation.cfm?id=982792.982856.

[31] Conrado Martínez, Daniel Panario, and Alfredo Viola. 2010. Adaptive sampling strategies for Quickselects. *ACM Trans. Algorithms* 6, 3 (2010), 53:1–53:45. DOI: https://doi.org/10.1145/1798596.1798606

[32] Conrado Martínez and Salvador Roura. 2001. Optimal sampling strategies in Quicksort and Quickselect. *SIAM J. Comput.* 31, 3 (2001), 683–705.

[33] David R. Musser. 1997. Introspective sorting and selection algorithms. *Software—Practice and Experience* 27, 8 (1997), 983–993.

[34] Charles Price. 1995. MIPS IV Instruction Set. Retrieved December 24, 2018 from http://math-atlas.sourceforge.net/devel/assembly/mips-iv.pdf
[35] Peter Sanders and Sebastian Winkel. 2004. Super scalar sample sort. In *Proceedings of Algorithms (ESA'04), Lecture Notes in Computer Science*, Susanne Albers and Tomasz Radzik (Eds.), Vol. 3221. Springer, Berlin, 784–796. DOI: https://doi.org/10.1007/978-3-540-30140-0_69
[36] Robert Sedgewick. 1977. The analysis of Quicksort programs. *Acta Inf.* 7, 4 (1977), 327–355.
[37] Robert Sedgewick. 1978. Implementing Quicksort programs. *Commun. ACM* 21, 10 (1978), 847–857.
[38] Sebastian Wild and Markus E. Nebel. 2012. Average case analysis of Java 7's dual pivot Quicksort. In *Proceedings of ESA'12, Lecture Notes in Computer Science*, Leah Epstein and Paolo Ferragina (Eds.), Vol. 7501. Springer, Berlin, 825–836. DOI: https://doi.org/10.1007/978-3-642-33090-2_71
[39] Sebastian Wild, Markus E. Nebel, and Ralph Neininger. 2015. Average case and distributional analysis of dual-pivot Quicksort. *ACM Trans. Algorithms* 11, 3 (2015), 22:1–42.
[40] John W. J. Williams. 1964. Algorithm 232: HEAPSORT. *Commun. ACM* 7, 6 (1964), 347–348.
[41] Vladimir Yaroslavskiy. 2009. Dual-Pivot Quicksort algorithm. Retrieved December 24, 2018 from http://codeblab.com/wp-content/uploads/2009/09/DualPivotQuicksort.pdf

Received May 2017; revised January 2018; accepted August 2018